

# John The Ripper

## Install John the Ripper

```
(kali㉿kali)-[/]
PS> sudo apt install john -y
Upgrading:
john john-data

Summary:
Upgrading: 2, Installing: 0, Removing: 0, Not Upgrading: 1402
Download size: 37.6 MB
Space needed: 510 kB / 44.3 GB available +

Get:1 http://http.kali.org/kali kali-rolling/main amd64 john amd64 1.9.0-Jumbo-1+git20211102-0kali11 [14.9 MB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 john-data all 1.9.0-Jumbo-1+git20211102-0kali11 [22.8 MB]
Fetched 37.6 MB in 52s (730 kB/s)
(Reading database... 426477 files and directories currently installed.)
Preparing to unpack .../john_1.9.0-Jumbo-1+git20211102-0kali11_amd64.deb...
Unpacking john (1.9.0-Jumbo-1+git20211102-0kali11) over (1.9.0-Jumbo-1+git20211102-0kali10)...
Preparing to unpack .../john-data_1.9.0-Jumbo-1+git20211102-0kali11_all.deb...
Unpacking john-data (1.9.0-Jumbo-1+git20211102-0kali11) over (1.9.0-Jumbo-1+git20211102-0kali10)...
Setting up john-data (1.9.0-Jumbo-1+git20211102-0kali11)...
Setting up john (1.9.0-Jumbo-1+git20211102-0kali11)...
Processing triggers for wordlists (2026.2.0)...
Processing triggers for kali-menu (2026.2.6)...
Processing triggers for man-db (2.13.1-1)...
```

## Create 5 deliberate weak passwords

```
(kali㉿kali)-[/]
PS> printf "password" | md5sum
5f4dcc3b5aa765d61d8327deb882cf99 -

(kali㉿kali)-[/]
PS> printf "admin" | md5sum
21232f297a57a5a743894a0e4a801fc3 -

(kali㉿kali)-[/]
PS> printf "letmein" | md5sum
0d107d09f5bbe40cade3de5c71e9e9b7 -

(kali㉿kali)-[/]
PS> printf "qwerty" | md5sum
d8578edf8458ce06fbc5bb76a58c5ca4 -

(kali㉿kali)-[/]
PS> printf "welcome" | md5sum
40be4e59b9a2a2b5dfffb918c0e86b3d7 -
```

Now create a file to store these MD5 hashes

```
(kali㉿kali)-[/home/kali]
PS> nano MD5_hashes.txt
```

Store the hashes in the file



The screenshot shows the nano text editor interface. The title bar indicates the file is 'MD5hashes.txt \*'. The editor content displays five lines of MD5 hashes: 5f4dcc3b5aa765d61d8327deb882cf99, 21232f297a57a5a743894a0e4a801fc3, 0d107d09f5bbe40cade3de5c71e9e9b7, d8578edf8458ce06fbc5bb76a58c5ca4, and 40be4e59b9b1e8a3c2c7b7b5e4b5e5b5. The bottom status bar shows various keyboard shortcuts for editing and navigation.

```
Session Actions Edit View Help
GNU nano 9.0 MD5hashes.txt *
5f4dcc3b5aa765d61d8327deb882cf99
21232f297a57a5a743894a0e4a801fc3
0d107d09f5bbe40cade3de5c71e9e9b7
d8578edf8458ce06fbc5bb76a58c5ca4
40be4e59b9b1e8a3c2c7b7b5e4b5e5b5
```

Cat MD5\_hashes.txt to check the content of the file

```
(kali㉿kali)-[/home/kali]
PS> cat MD5_hashes.txt
5f4dcc3b5aa765d61d8327deb882cf99
21232f297a57a5a743894a0e4a801fc3
0d107d09f5bbe40cade3de5c71e9e9b7
d8578edf8458ce06fbc5bb76a58c5ca4
40be4e59b9b1e8a3c2c7b7b5e4b5e5b5
```

# Crack the hashes with John the Ripper

```
(kali㉿kali)-[/home/kali]
PS> john --format=raw-md5 MD5_hashes.txt
Using default input encoding: UTF-8
Loaded 5 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press Ctrl-C to abort, or send SIGUSR1 to john process for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
password      (?)
qwerty         (?)
letmein        (?)
admin          (?)
Proceeding with incremental:ASCII

y
password
admin
letmein
qwerty
4g 0:00:04:27 3/3 0.01492g/s 4158Kp/s 4158Kc/s 4158KC/s liczkua..licztld
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session aborted
```

John successfully loaded all hashes

Lets check what passwords are cracked

```
(kali㉿kali)-[/home/kali]
PS> john --show --format=raw-md5 MD5_hashes.txt
?:password
?:admin
?:letmein
?:qwerty

4 password hashes cracked, 1 left
```

4 password hashes has been cracked, 1 left.

In this lab, I practiced using John the Ripper to crack five MD5 password hashes on Kali Linux. John successfully recovered four out of five passwords using password-cracking techniques and wordlists. One hash remained uncracked, demonstrating that not every password can be recovered using a limited wordlist or within a short period.

The lab showed how weak and commonly used passwords can be cracked relatively easily and reinforced the importance of strong, unique passwords and secure password hashing with salting. It also helped me understand that cracking results depend on factors such as password complexity, available wordlists, computing resources, and attack time. Overall, the exercise provided practical experience with password security and the limitations of automated cracking tools.

## Hashcat

Make sure you have the file rockyou.txt

```
(root@kali)-[~]
# ls -lh /usr/share/wordlists/rockyou.txt
-rw-r--r-- 1 root root 134M May  7 06:16 /usr/share/wordlists/rockyou.txt
```

Then create a file WPA2 test hash

And check the content of the file

```
(root@kali)-[~]
# cat wpa-test.hc22000
WPA*01*4d4fe7aac3a2cecab195321ceb99a7d0*fc690c158264*f4747f87f9f4*686173686361742d6573736964**
```

Now run Hashcat with rockyou.txt

```
(root@kali)-[~]
# hashcat -O 1 -m 22000 wpa-test.hc22000 /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 7.1+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 21.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-5200U CPU @ 2.20GHz, 738/1477 MB (738 MB allocatable), 2MCU

Minimum password length supported by kernel: 8
Maximum password length supported by kernel: 63
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Hash
* Single-Salt
* Slow-Hash-SIMD-LOOP

Watchdog: Temperature abort trigger set to 90c

* Device #1: Not enough allocatable device memory or free host memory for mapping.

Started: Fri Oct  2 07:57:57 2026
Stopped:  Fri Oct  2 07:58:05 2026
```

Hashcat could not complete the command due to low memory allocation.

**WPA2 Hashcat Exercise:** A WPA2-style test hash was loaded using Hashcat mode 22000 and the rockyou.txt wordlist. Hashcat initially lacked an OpenCL runtime, which was resolved by installing PoCL. Hashcat subsequently detected the Kali VM's Intel CPU, but the attack could not proceed because the VM had insufficient allocatable memory. The exercise demonstrated WPA2 hash format recognition, Hashcat configuration, wordlist-based cracking, and troubleshooting of resource limitations.

## Reflection Notes: Why Strong Passwords + Salting Are Essential

This lab helped me understand why password security depends on more than simply hiding passwords from users. Weak passwords such as password, admin, and qwerty can be recovered quickly using password-cracking tools like John the Ripper. This demonstrated how attackers can use wordlists and automated tools to test thousands or millions of password guesses.

Stron

g passwords are therefore essential because they make guessing and brute-force attacks significantly more difficult. A long, unique password with a combination of words, numbers, and symbols provides much more resistance than a short or commonly used password.

I also learned that salting is critical when storing passwords. A salt is a unique random value added to each password before it is processed by a password-hashing algorithm. This means that even if two users choose the same password, their stored password hashes will be different. Salting also makes precomputed lookup tables, such as rainbow tables, much less useful.

The biggest lesson from this exercise is that strong passwords protect against guessing, while proper salted password hashing protects stored credentials if the database is compromised. Together, they provide an important layer of defense against credential theft and password-cracking attacks.